Vom Netz-Konzept zum ersten Kontakt

Ein erster Rust/egui-Prototyp für den bewussten Austausch von Chat, Code und Prompts zwischen zwei Geräten.

Worum geht es?

Die vorherige Übersicht „copi-Netz-Konzept und Empfehlungen“ vom 13.10.2026 beschreibt ein privates Netz für Seminare und freie Projekte: P2P über iroh, verschlüsselte Gruppen über MLS, optionaler UdK-Geräteausweis über authentik. Diese neue Übersicht dokumentiert den ersten tatsächlich gebauten Schritt – nicht die Fertigstellung des Konzepts.

Was aus dem Konzept erhalten bleibt

- Agentenchat in der Mitte, Netz rechts, türkis als eigenes Signal.
- Agentenstatus bleibt direkt über der Eingabe.
- Keine fremde Fernsteuerung, keine automatische Promptausführung.
- Frontendunabhängiger Rust-Kern, später auch Tauri/WebKit.

> Jetzt möglich: direkter Kontakt zwischen zwei Geräten. Noch nicht möglich: gemeinsames Seminar-/Projektnetz mit MLS, authentik und Relay.

Repo und Branch

github.com/KGasteier/pi-lite · Branch cocopi

Erster Code-Commit: b3642ba. Anleitung und Installer werden mit dieser Übersicht ergänzt. Keine fremden Zugangsdaten erforderlich; individuelle KI-Zugänge bleiben lokal.

1. Zwei Testpersonen vorbereiten

Am einfachsten: zwei Linux-Rechner im selben vertrauenswürdigen LAN, ohne WLAN-Client-Isolation. Zunächst nur erfundene Texte verwenden. Keine vertraulichen Arbeiten und keine echten Seminargruppen. Jeder hat seine eigene lokale pi-Installation/Konfiguration; Schlüssel oder private Sessions niemals austauschen.

Voraussetzungen: Git, Internet für Rust-Abhängigkeiten, aktuelle stabile Rust-Toolchain (iroh benötigt mindestens Rust 1.91), Linux-Desktop mit Wayland oder X11 und native Build-Werkzeuge. Genügend freier Speicher für mehrere GB Cargo-Buildartefakte. Auf Omarchy/Arch ggf. `sudo pacman -S --needed base-devel rustup pkgconf`; eine vorhandene Rust-Installation nicht blind ersetzen. Auf anderen Distributionen entsprechende C-/Grafik-Buildpakete installieren; kein pauschaler Systeminstaller. Der Build wurde nur auf Linux aarch64 geprüft, x86_64/macOS/Windows nicht live verifiziert.

Für den Agentenchat: pi nach dessen eigener Installationsanleitung einrichten und `pi --version` prüfen. Der Direkttransport ist unabhängig vom KI-Zugang; ohne pi funktioniert der Agentenchat nicht. **Nicht im COPI_DEMO-Modus testen:** dort ist der Transportstart absichtlich deaktiviert.

2. Aus dem öffentlichen Repo installieren

```
git clone --branch cocopi --single-branch https://github.com/KGasteier/pi-lite.git
cocopi-src
cd cocopi-src
rustc --version
CARGO_BUILD_JOBS=2 cargo test --locked --workspace --features cocopi --manifest-path
copi/Cargo.toml
bash copi/scripts/install-cocopi.sh
~/.local/bin/cocopi
```

Erster Build benötigt Internet; er kann mehrere Minuten dauern. Das Skript installiert nur die separate Binary, verändert weder copi noch pi-lite und richtet keinen Autostart ein. Bei kleinen Rechnern `CARGO_BUILD_JOBS=1` verwenden. Ein Buildfehler ist kein Anlass für ein blindes Systemupdate; vollständige Fehlermeldung und Toolchainversion teilen, aber keine Tokens/Sessioninhalte.

3. Direkten Kontakt herstellen

1. Beide starten cocopi aus dem Terminal. Netz ist rechts sichtbar, sonst türkisen Schalter oben rechts anklicken. Agentenstatus bleibt über der Eingabe.

2. Beide klicken **Direkttransport ausdrücklich starten**. Es werden lokale Netzwerk-Sockets geöffnet; keine öffentlichen Relays/Discovery verwendet.

3. Kurz warten, dann **Adresse kopieren**. Kontaktadresse enthält Geräteschlüssel und Netzwerkadressen, möglicherweise IP-Adressen. Nicht öffentlich posten.

4. Person A übermittelt die vollständige Adresse an B, B ihre an A – über einen vertrauenswürdigen Kanal oder persönlich. Den vollständigen Geräteschlüssel innerhalb der Adresse unabhängig vergleichen. Noch keine Sicherheitswörter und keine Einmal-Einladung.

5. Beide fügen die erhaltene JSON-Adresse unverändert ins Kontaktfeld ein und klicken **Diesen Kontakt ausdrücklich bestätigen**.

6. Die Meldung „Kontakt gewählt“ bedeutet nur lokale Auswahl, noch keinen gelungenen Verbindungsaufbau. Erst bestätigter Empfang beim Senden weist eine Verbindung nach.

Schlüssel sind bei jedem Transportstart neu. Nach Stoppen/Neustart beide Adressen erneut austauschen. Ein einziges bestätigtes Gegenüber pro App, keine Gruppe.

4. Austausch nachweisen

- A wählt Chat und schreibt `Test von A – keine vertraulichen Daten`.
- **Senden prüfen …**, Inhalt prüfen, dann **Jetzt übertragen**.
- B muss eine empfangene Karte sehen; A muss „Empfang auf Gegenseite bestätigt“ sehen.
- B antwortet genauso. Nur wenn beide Richtungen funktionieren, ist der Direktkontakt erfolgreich.
- Danach Code mit harmlosem Text testen. Er wird nur angezeigt/kopiert, niemals ausgeführt.
- Prompt testen: B klickt **In Eingabe übernehmen …**, bestätigt die Ersetzung der aktuellen Eingabe. Der Text muss editierbar sein und darf nicht automatisch an pi/KI gehen. Für diesen Transporttest nicht auf Agenten-Senden klicken.
- Beide stoppen den Transport. Keine Wiederverbindung ohne erneute Adressbestätigung erwarten.

5. Was derzeit kein Netzwerk aufbaut

Lokale Projekte, Tutorialschritte und Handheben sind nur UI-/Ablaufentwürfe und werden nicht synchronisiert. Mehrere Personen können paarweise testen; das ist kein zusammenhängendes Gruppennetz. Keine Dateien, kein Offline-Postfach, keine MLS-Epochen, kein authentik-Ausweis. Seminare sind bis zur verifizierten Lehrendenrolle gesperrt.

6. Fehlersuche / Abnahme

Bei Zeitlimit: laufen beide Transportseiten, sind die aktuell kopierten Adressen vollständig, haben beide bestätigt, sind beide im selben LAN, ist WLAN-Isolation aktiv, blockiert eine lokale Firewall direkte UDP-Verbindungen? Keine Firewall komplett deaktivieren und keine Ports ins öffentliche Internet freigeben. UDP-Bindings sind im Prototyp dynamisch; gezielte lokale Freigabe mit Administration abstimmen. Fehlende LAN-Erreichbarkeit lässt sich mit diesem Build nicht über Relay umgehen.

Testnotiz gemeinsam festhalten: Commit (`git rev-parse --short HEAD`), OS/CPU, Rustversion, LAN-Aufbau, beide Richtungen erfolgreich?, Prompt nur übernommen?, Stop/Neupaarung funktioniert? Keine Kontaktadressen, IPs, Schlüssel oder private Screenshots ins öffentliche Repo.

7. Von zwei Personen zum Seminar-/Projektnetz

Zuerst diesen LAN-Test abnehmen. Dann Entwicklung: dauerhafte Geräteschlüssel/Keychain → eigener Relay-Adapter und Relay-only → OpenMLS und sichere Einladungen/Gruppen → signierter authentik-Ausweis → synchronisierte Projekt-/Tutorialereignisse. Parallel Server/DNS/TLS, authentik-Rollen, Datenschutz und Review organisieren; siehe Deployment-Anleitung. Erst danach freiwilligen Pilotkurs starten.

Updates mit sauberem Arbeitsstand im Branch cocopi:

```
bash copi/scripts/update-cocopi.sh
bash copi/scripts/install-cocopi.sh --skip-build
```

Das Update-Skript verwendet für den Fetch `gh auth git-credential` (lokale gh-Anmeldung erforderlich). Ohne gh: `git fetch https://github.com/KGasteier/pi-lite.git copi && git merge --no-edit FETCH_HEAD`, danach Tests und Build wie oben. Nicht `copi/main` durch cocopi ersetzen. Auch cocopi-eigene Aktualisierungen holen: `git pull --ff-only origin cocopi`; Konflikte vor dem Bauen prüfen. Laufende Fenster selbst regulär neu starten.

Server, authentik und KI-Zugang

Serverbetrieb

Test-VM, DNS/TLS und Zuständigkeit festlegen. Eigenes iroh-Relay ohne Inhaltsspeicher betreiben; kurze Logs, Limits und Updates. Client-Adapter dafür fehlt noch. Campus-/Heimnetz und Relay-only separat testen.

authentik und Geräteausweis

Nativer öffentlicher OIDC-Client mit PKCE S256, kein Client-Secret im Binary. Lehrendenrolle serverseitig zuweisen. Ein eigener Ausweis-Dienst prüft Login und Gerätebesitz und signiert kurzlebige Geräteausweise. Trust-Roots, Schlüsselrotation und Widerruf definieren. Der Prototyp implementiert dies noch nicht.

api.udk.digital

KI-Zugang unabhängig vom Netz über den vorhandenen pi-Prozess einrichten. Betreiber bestätigt API-Protokoll und Modell-IDs, jede Person erhält einen begrenzten Token. /v1/models verlangt Authentifizierung; Streaming und Tool Calls sind noch nicht mit Zugang getestet. OIDC-Token nicht mit KI-API-Token verwechseln.

```
baseUrl: https://api.udk.digital/v1
api: openai-completions (nur wenn bestätigt)
apiKey: ${UDK_API_KEY}
model: vom Betreiber bestätigte ID
```

Vollständiges privates models.json-Beispiel und Sicherheitsanforderungen: docs/COCOPI_DEPLOYMENT.md. Keine vorhandene Konfiguration ersetzen, keine Tokens ins Repo oder Netz teilen.

Freigabe

Datenschutz, freiwillige Teilnahme, Löschfristen und unabhängigen Sicherheitsreview organisieren. Kein vertraulicher Lehreinsatz vor Abschluss der fehlenden Schutzmechanismen.

Was wir nach dem Test wissen

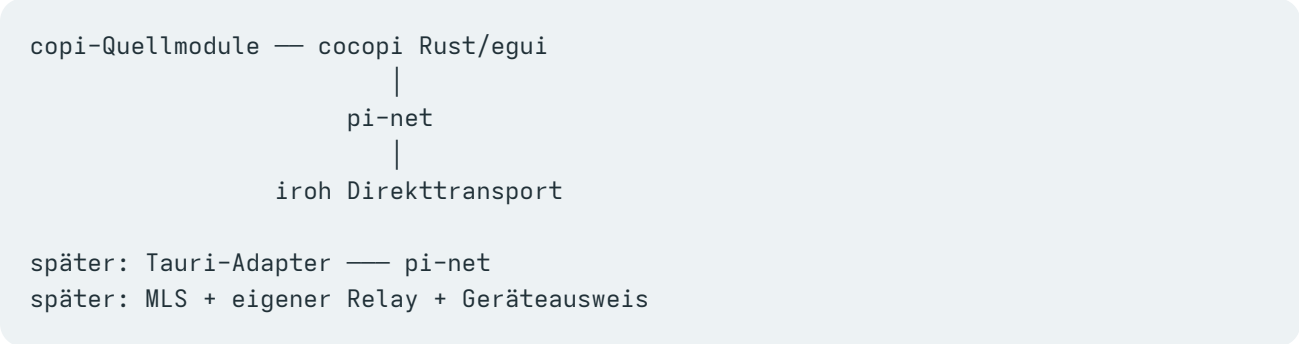
Erfolgskriterien

- Beide Geräte haben aktuelle Kontaktadressen bestätigt.
- Chat in beide Richtungen, jeweils Empfang bestätigt.
- Code bleibt Daten, keine Ausführung.
- Prompt wird nur nach Zustimmung editierbar übernommen, nicht gesendet.
- Stoppen und Neupaarung nach Neustart verstanden.

Bereits geprüft

Workspace-Tests, echte iroh-Übertragung zwischen zwei Endpunkten auf derselben Maschine, Release-Build und identische Installationsprüfsumme. Headless-Layout geprüft. Noch kein Zwei-Rechner-, WAN-/Relay- oder UI-Livetest.

Architektur



Weiterführende Unterlagen im Branch

- docs/COCOPI_QUICKSTART.md – ausführbarer Testablauf
- docs/COCOPI.md – Architektur, Stand und Grenzen
- docs/COCOPI_DEPLOYMENT.md – externe Einrichtung
- copi/scripts/install-cocopi.sh – separate Installation
- copi/scripts/update-cocopi.sh – expliziter copi-Abgleich

Git übernimmt copi-Neuerungen nicht von selbst: dieselben Module vermeiden den UI-Fork, der Merge übernimmt neue Commits. Kein stiller Austausch laufender Apps.